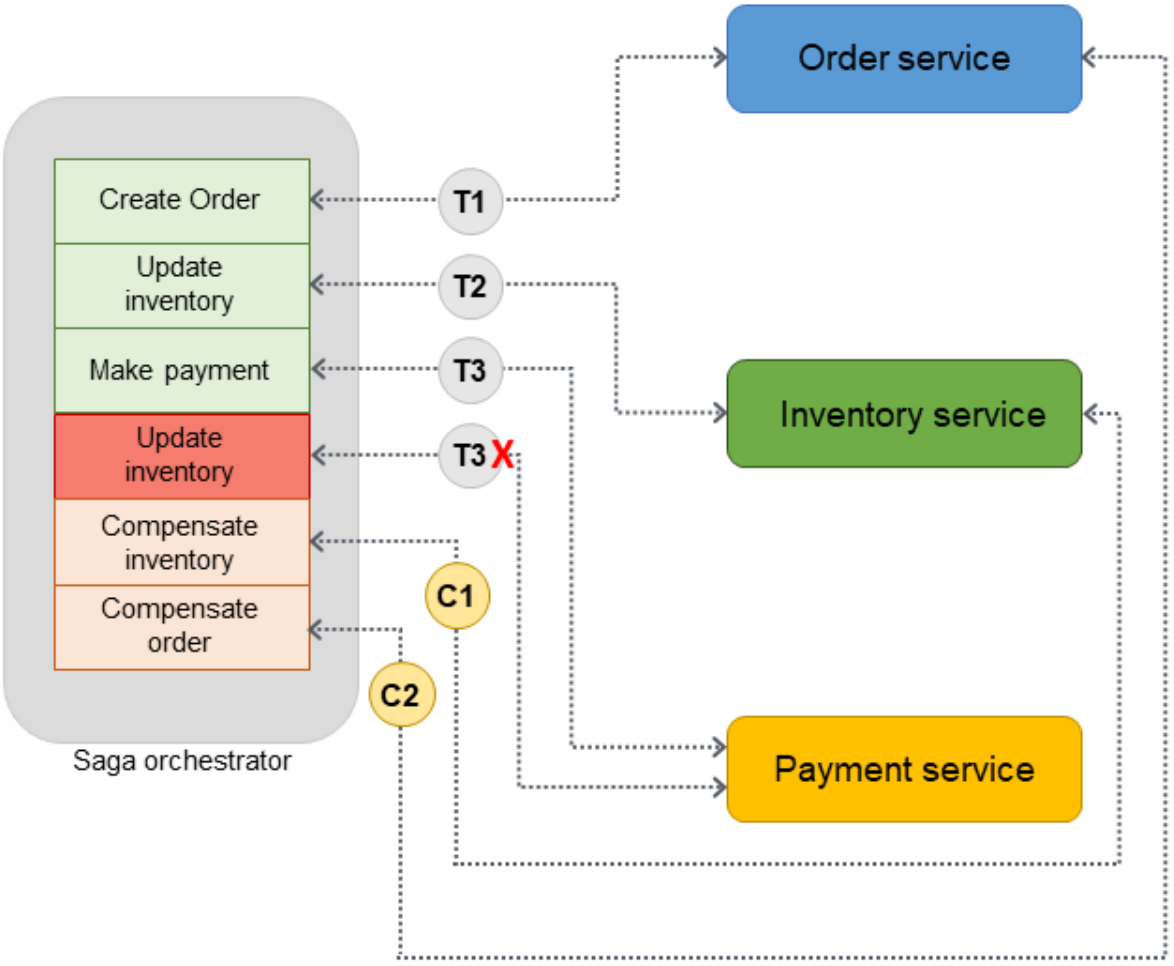
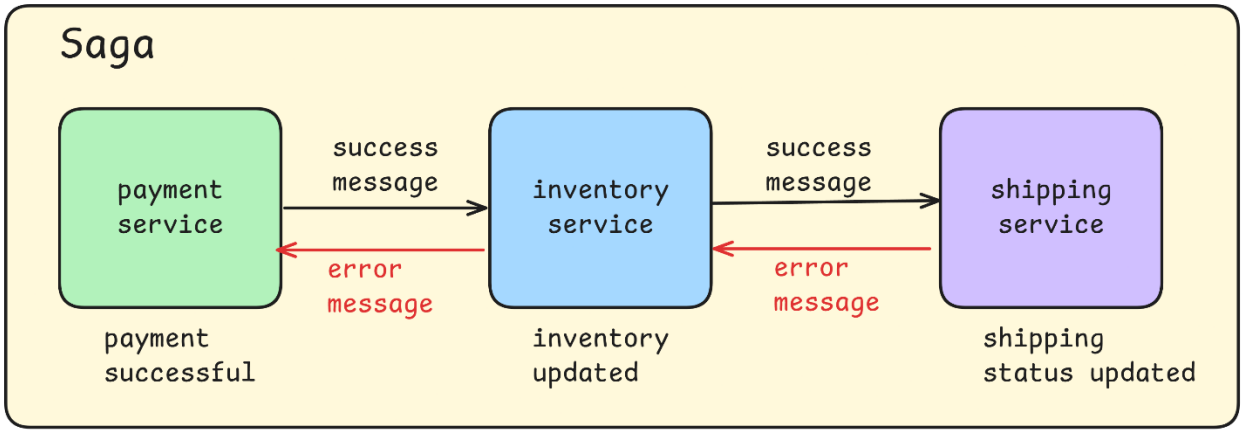
Below is a **practical, step-by-step implementation of the Saga Pattern in ASP.NET Core**, using a **realistic Order–Payment–Inventory workflow**.

I will use an **Orchestration-based Saga** because it is:

- Easier to understand
- Easier to debug
- Common in enterprise systems

---






---

**1. Architecture Overview**

**Services Involved**

| **Service** | **Responsibility** |
|---|---|
| OrderService | Create / cancel order |
| PaymentService | Charge / refund payment |
| InventoryService | Reserve / release stock |
| OrderSagaOrchestrator | Controls saga flow |

Each service:

- Owns its database
- Exposes HTTP endpoints
- Performs **local transactions only**

---

**2. Saga Flow (Business Logic)**

**Happy Path**

1. Create Order (Pending)
2. Charge Payment

3. Reserve Inventory
4. Mark Order as Completed

**Failure Path**

- If Payment fails → Cancel Order
- If Inventory fails → Refund Payment + Cancel Order

---

**3. Shared DTOs (Contracts)**

Create a **Shared.Contracts** project.

```
public record CreateOrderRequest(int ProductId, int Quantity, decimal Amount);

public record OrderResponse(int OrderId);


public record PaymentRequest(int OrderId, decimal Amount);

public record InventoryRequest(int OrderId, int ProductId, int Quantity);
```

---

**4. Order Service**

**Order Entity**

```
public class Order
{
   public int Id { get; set; }
   public string Status { get; set; } // Pending, Completed, Cancelled
}
```

**Controller**

```
[ApiController]
[Route("api/orders")]
public class OrderController : ControllerBase
{
```

```
    private static readonly List<Order> _orders = new();

    [HttpPost]
    public IActionResult CreateOrder()
    {
        var order = new Order { Id = _orders.Count + 1, Status = "Pending" };
        _orders.Add(order);
        return Ok(new { order.Id });
    }

    [HttpPost("{id}/cancel")]
    public IActionResult CancelOrder(int id)
    {
        var order = _orders.First(o => o.Id == id);
        order.Status = "Cancelled";
        return Ok();
    }

    [HttpPost("{id}/complete")]
    public IActionResult CompleteOrder(int id)
    {
        var order = _orders.First(o => o.Id == id);
        order.Status = "Completed";
        return Ok();
    }
}
```

---

**5. Payment Service**

**Controller**

```
[ApiController]
[Route("api/payments")]
public class PaymentController : ControllerBase
{
    [HttpPost("charge")]
    public IActionResult Charge(PaymentRequest request)
    {
        // Simulate payment failure randomly
        if (request.Amount > 5000)
            return BadRequest("Payment failed");

        return Ok();
    }

    [HttpPost("refund")]
    public IActionResult Refund(PaymentRequest request)
    {
        return Ok();
    }
}
```

---

**6. Inventory Service**

**Controller**

```
[ApiController]
[Route("api/inventory")]
public class InventoryController : ControllerBase
{
    [HttpPost("reserve")]
    public IActionResult Reserve(InventoryRequest request)
    {
        if (request.Quantity > 10)
            return BadRequest("Insufficient stock");

        return Ok();
    }

    [HttpPost("release")]
    public IActionResult Release(InventoryRequest request)
    {
        return Ok();
    }
}
```

---

**7. Saga Orchestrator (Core Part)**

**Saga Orchestrator Controller**

```
[ApiController]
[Route("api/saga")]
public class OrderSagaController : ControllerBase
{
```

```
private readonly HttpClient _http;

public OrderSagaController(HttpClient http)
{
    _http = http;
}

[HttpPost("place-order")]
public async Task<IActionResult> PlaceOrder(CreateOrderRequest request)
{
    // STEP 1: Create Order
    var orderResponse = await _http.PostAsJsonAsync(
        "https://orderservice/api/orders", new { });

    var order = await orderResponse.Content.ReadFromJsonAsync<OrderResponse>();

    try
    {
        // STEP 2: Payment
        var paymentResponse = await _http.PostAsJsonAsync(
            "https://paymentservice/api/payments/charge",
            new PaymentRequest(order!.OrderId, request.Amount));

        paymentResponse.EnsureSuccessStatusCode();

        // STEP 3: Inventory
```

```
            var inventoryResponse = await _http.PostAsJsonAsync(
                "https://inventoryservice/api/inventory/reserve",
                new InventoryRequest(order.OrderId, request.ProductId, request.Quantity));

            inventoryResponse.EnsureSuccessStatusCode();

            // STEP 4: Complete Order
            await _http.PostAsync(
                $"https://orderservice/api/orders/{order.OrderId}/complete", null);

            return Ok("Order completed successfully");
        }
        catch
        {
            // Compensation logic
            await _http.PostAsJsonAsync(
                "https://paymentservice/api/payments/refund",
                new PaymentRequest(order!.OrderId, request.Amount));

            await _http.PostAsync(
                $"https://orderservice/api/orders/{order.OrderId}/cancel", null);

            return BadRequest("Order failed. Compensation executed.");
        }
    }
}
```

---

**8. Register HttpClient**

builder.Services.AddHttpClient();

---

**9. What Happens Internally (Very Important)**

| **Step** | **Type** |
|---|---|
| Order creation | Local transaction |
| Payment charge | Local transaction |
| Inventory reserve | Local transaction |
| Compensation | Reverse local transactions |
| No distributed transaction | ✓ |

---

**10. Key Production Improvements**

| **Area** | **Improvement** |
|---|---|
| Reliability | Retry + Circuit Breaker |
| Messaging | Replace HTTP with events |
| State | Persist saga state |

---

**11. What You Have Learned**

- How Saga replaces distributed transactions
- How compensation works in real code
- Why orchestration is easier for beginners
- How services remain loosely coupled
- How failure is handled gracefully